

CSTA / Technical design

Library Design and Scientific Use

From numerical primitives to Wang–Landau sampling

Statistical mechanics and computational physics

Workspace version	3.0.0
Language	Rust 2024; minimum Rust 1.98.0
Document date	6 September 2026
Scope	Current workspace, including local implementation changes
License metadata	GPL-3.0-or-later

This document describes implemented behavior. It separates mathematical models, API contracts, numerical range, and scientific limitations. Example budgets illustrate usage; they do not certify convergence.

Reading this document

Start with Chapters 1–2 for the workspace and physical conventions. Chapters 3–8 cover primitives, random construction, canonical sampling, and analysis. Chapter 9 describes the separate dynamics API. Chapters 10–15 cover energy support, serial and parallel DOS estimation, and reweighting. Persistence and validation follow.

The code is the authority for current behavior. Source paths at the end of each chapter locate the owning implementation; the source map in Appendix A covers all six crates. The design reflects the local workspace on the document date, including changes not yet committed. The package version does not assert that this workspace has been published to crates.io.

Complete Rust examples are embedded in this file and can be extracted and run with `docs/check_examples.py`. Their dependency is the local `csta` facade. Smaller trait and configuration excerpts are identified as excerpts. The build instructions in `docs/README.md` explain PDF generation and example validation. No external images, syntax-highlighting service, or shell escape is needed to build the document.

Three distinctions recur throughout the library. A completed run means that a budget was fulfilled. A numerical error means that an operation could not be represented or validated. A statistical uncertainty estimate describes sampling variation under assumptions about stationarity and dependence. None of these alone establishes physical correctness or adequate exploration.

Contents

Reading this document	i
1 Workspace architecture	1
1.1 Purpose and scope	1
1.2 Public entry points and dependencies	1
2 Physical and numerical conventions	3
2.1 Units and ensembles	3
2.2 Floating-point range and errors	3
2.3 What a seed controls	4
3 Core vectors and geometry	5
3.1 Small floating vectors	5
3.2 Periodic coordinates and nearest images	5
3.3 Lattice topology	6
4 Random construction	7
4.1 Trait and iterator	7
4.2 Deriving random models	7
5 The model contract	9
5.1 State, parameters, and reversible changes	9
5.2 Optional hooks	9
6 Canonical sampling and observations	12
6.1 Transition and rollback	12
6.2 Budgets and observation timing	12
6.3 Observer interfaces	13
7 Lattice models and temperature exchange	14
7.1 Two-dimensional Ising model	14

7.2	Lattice gas	14
7.3	Temperature replica exchange	15
8	Statistics and finite-size analysis	16
8.1	Streaming moments	16
8.2	Blocking and autocorrelation	16
8.3	Thermodynamic summaries and nonlinear errors	17
8.4	Finite-size helpers	17
9	Deterministic dynamics	19
9.1	A separate NVE interface	19
9.2	Energy monitoring and force helpers	19
10	Energy grids and DOS data	21
10.1	Explicit support	21
10.2	Mass, density, and normalization	22
10.3	Data lifecycle and counters	22
11	Serial Wang–Landau and SAMC	23
11.1	Algorithm composition	23
11.2	Preliminary refinement	23
11.3	Production schedule and bounded execution	23
11.4	Diagnostics and stopping semantics	25
11.5	Incremental sessions	25
12	DOS thermodynamics and reference spectra	26
12.1	Canonical reconstruction	26
12.2	Microcanonical quantities	27
12.3	Reference models and exact ranges	27
13	Parallel energy-window sampling	28
13.1	Window and replica layout	28
13.2	Chunk execution and exchanges	28
13.3	Averaging and joining DOS fragments	29
13.4	Parallel sessions and practical limits	30
14	Observables, reweighting, and DOS uncertainty	31
14.1	Conditional observable moments	31
14.2	Single-histogram canonical reweighting	32

14.3 Independent DOS experiments	33
15 Sparse joint density of states	35
15.1 Coordinates and field reweighting	35
15.2 Joint sampling through the existing engine	35
16 Persistence and reproducible continuation	37
16.1 Opt-in serialization	37
16.2 File format and validation	38
16.3 What continuation preserves	38
17 Validation, cost, and design boundaries	39
17.1 Error and partial-result policy	39
17.2 Operation and memory costs	39
17.3 Existing verification strategy	40
17.4 Scientific validation beyond API correctness	41
17.5 Deliberate boundaries and extension rules	42
A Source and example map	43
A.1 Owning implementation files	43
A.2 Runnable repository examples	44
B Example and document reproducibility	45
References	46

Chapter 1

Workspace architecture

1.1 Purpose and scope

CSTA provides composable Rust components for small statistical-mechanics simulations and numerical experiments. The ordinary workflow is explicit: construct a model, choose a seeded random generator, set a finite budget, run a sampler, and analyze retained observations. Advanced capabilities are added by choosing a module or enabling persistence. Basic model implementations retain four required **State** methods.

The workspace includes canonical Metropolis–Hastings sampling, temperature replica exchange, Wang–Landau preliminary refinement, SAMC production, and parallel energy-window exchange. It also includes lattice models, statistical estimators, DOS thermodynamics, and a small NVE integrator. It is neither a complete molecular-simulation engine nor a general linear-algebra package.

Crate	Responsibility
<code>csta_core</code>	Floating vectors, periodic geometry, lattice connectivity, fixed-step dynamics, optional checkpoint file infrastructure.
<code>csta_montecarlo</code>	Randomizable , an infinite sampling iterator, Gaussian draws, sphere directions, optional checkpoint RNG.
<code>csta_derive</code>	Procedural derive for random struct and enum construction.
<code>csta_metropolis</code>	State , canonical transitions, observers, lattice models, statistics, finite-size helpers, temperature replicas.
<code>csta_wl</code>	Validated energy grids, DOS data, WL/SAMC, energy-window replicas, reference models, analysis, joint DOS, and sessions.
<code>csta</code>	Public facade, prelude, integration tests, and runnable examples.

Table 1.1: Ownership follows scientific responsibility.

1.2 Public entry points and dependencies

`csta` re-exports vector types, canonical types, and random utilities. Geometry and dynamics remain modules; Wang–Landau is `csta::wl`. The prelude additionally imports observer traits. Explicit imports are helpful in larger programs: a glob import includes the canonical **Result**<T> alias, which has one type parameter. Use `std::result::Result`<T, E> when a different error type is needed.

Direct use of an owning crate is supported. Runtime dependencies must continue pointing from

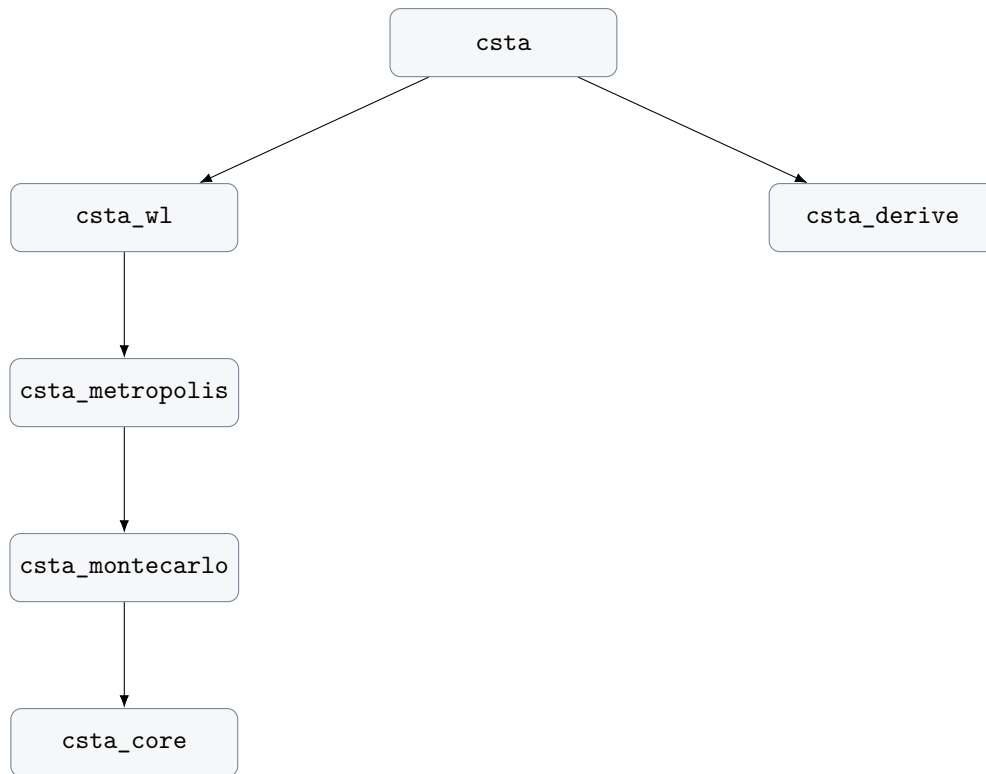


Figure 1.1: Main dependency spine; arrows point to dependencies. Additional direct edges let the facade re-export lower layers and let WL and Metropolis use core utilities. The derive crate emits code that refers to the facade; it does not introduce a runtime dependency cycle.

algorithms toward primitives; the lower crates do not depend on `csta`. The procedural macro generates references to `csta` and `rand`, so consumers of the derive need those dependency names available.

```

[package]
name = "physics-experiment"
version = "0.1.0"
edition = "2024"
rust-version = "1.98.0"

[dependencies]
csta = { path = "../csta2/csta" }
rand = "=0.10.2"
  
```

Listing 1.1: Dependency excerpt for a separate project. Adjust the local path.

Default features are empty. `serde` enables core serialization support. `checkpoint` enables the sampler persistence stack and the serializable PCG64 generator; it also enables the required Serde support. Ordinary runs do not need either feature. Dependency versions and path relationships are fixed by the workspace manifests and lockfile; toolchain requirements should not be reduced to fit an older local compiler.

Implementation: `Cargo.toml`.

Implementation: `csta/src/lib.rs`.

Implementation: `csta/src/prelude.rs`.

Chapter 2

Physical and numerical conventions

2.1 Units and ensembles

The APIs use bare floating-point numbers. They do not encode dimensions or convert units. Choose a consistent energy scale, length scale, mass scale, and time scale before constructing a model. The inverse temperature is always

$$\beta = \frac{1}{k_B T}. \quad (2.1)$$

Examples use reduced units with $k_B = 1$. A canonical transition targets weights proportional to $\exp(-\beta H)$. The lattice gas supplies $H - \mu N_p$ as its `State::energy`; this produces a grand-canonical distribution over binary site occupations. DOS sampling targets a distribution flattened in the supplied energy bins. Dynamics advances a deterministic trajectory under a force law.

Finite negative β is accepted by the generic canonical and DOS APIs. Its physical use requires a normalizable spectrum, generally bounded above. A truncated DOS always defines a finite supplied spectrum, even when the untruncated physical system would not admit that ensemble. Wolff updates have a more restrictive nonnegative- β domain.

Quantity	Convention
<code>State::energy</code>	Total effective Hamiltonian; not energy per site.
<code>M</code>	Signed total magnetization unless the caller deliberately defines another observable.
<code>ThermalSummary</code>	Energy, magnetization, heat capacity, and susceptibility per site; Binder cumulant dimensionless.
<code>WLData::specific_heat</code>	Total $C = k_B \beta^2 \text{Var}(E)$; divide by size externally.
<code>dos()</code>	Natural logarithm of a degeneracy or continuous-bin mass.
Canonical step	One attempted proposal, including rejection or a self-loop.
Wolff step	One cluster update, whose number of flipped spins varies.
Dynamics step	One time increment Δt ; unrelated to Monte Carlo time.

Table 2.1: Conventions that must remain explicit in data analysis.

2.2 Floating-point range and errors

Vectors have `f32` and `f64` variants; sampling energies and statistical analysis use `f64`. Finite inputs do not ensure that every product, difference, or sum is representable. Checked numerical

APIs reject nonfinite results where specified. Raw vector operators retain IEEE arithmetic, including infinities and NaNs. There is no arbitrary-precision fallback.

Logarithmic weights avoid direct exponentiation of large energies or DOS values. They reduce overflow risk, but do not make absolute partition functions or moments representable in every scale. DOS updates smaller than floating-point resolution are explicit errors. Changing units or removing an irrelevant DOS offset can be necessary before starting a new run.

Most validation failures are returned as **Result**. Undefined statistical estimates often use **Option** inside a successful result. These are different outcomes: **None** can mean insufficient data, while **Error** means invalid input or a failed numerical operation. Legacy sampling conveniences and infallible random construction can panic. Error boundaries are summarized in Chapter 17.

2.3 What a seed controls

An explicit RNG makes randomness part of the simulation state. Repeating a seed is meaningful only with the same algorithm, initial model, parameter values, proposal ordering, dependency implementation, and relevant platform behavior. Changing a callback, adding a random draw, or changing a chunk boundary can change the trajectory. A seed alone is insufficient experiment metadata.

Record the Hamiltonian, boundary conditions, size, units, seed/RNG, schedules, energy support, stopping reason, and software versions with results. Statistical error bars should be accompanied by the number of observations, correlation or block assumptions, and evidence from independent runs when available.

Chapter 3

Core vectors and geometry

3.1 Small floating vectors

`Vec2f32`, `Vec2f64`, `Vec3f32`, `Vec3f64`, `Vec4f32`, and `Vec4f64` are public tuple structs. They provide constructors, coordinate accessors, array/tuple conversions, precision conversions, arithmetic, dot products, lengths, distances, and normalization. They are `Copy`; optional Serde support serializes their coordinates.

Length uses chained hypotenuse calculations. Normalization first rescales by the largest absolute coordinate, which allows a finite direction to be recovered from many very large or very small vectors. `try_normalize` returns `None` for zero vectors or nonfinite coordinates. `normalize` returns the zero vector in those cases.

Squared length and dot products use direct arithmetic and can overflow even when a normalized direction exists. Distance subtracts coordinates before computing length; the subtraction itself can overflow. Narrowing from `f64` to `f32` can lose precision or become infinite. Scalar division, including `/=`, follows IEEE division. There are no checked arithmetic operators, matrices, cross-product API, or dimensioned physical quantities.

```
use csta::{Vec2f64, Vec3f64};

fn main() {
    let v = Vec3f64::new(3.0, 4.0, 0.0);
    assert_eq!(v.len(), 5.0);
    assert!((v.try_normalize().unwrap().len() - 1.0).abs()
        < 1e-14);
    assert!(Vec2f64::default().try_normalize().is_none());
    let huge = Vec2f64::new(f64::MAX, f64::MAX);
    assert!(huge.len().is_infinite());
    assert!(huge.try_normalize().is_some());
}
```

Listing 3.1: Vector arithmetic and explicit undefined directions.

3.2 Periodic coordinates and nearest images

`PeriodicBox<D>` represents an orthorhombic box with $D > 0$ finite, positive side lengths. `wrap` maps finite positions into $[0, L_d)$ for each coordinate. `displacement(from, to)` returns the nearest-image displacement in $[-L_d/2, L_d/2)$. Exact half-box ties choose the negative side. Both endpoints are wrapped before subtraction, reducing overflow risk for large unwrapped positions.

The box does not store particles, apply forces, or impose a cutoff. It has no triclinic cell matrix, shear convention, or long-range summation. The minimum image is a geometric operation; the validity of a periodic interaction model is a separate physical question.

```
use csta::geometry::{Boundary, Lattice2D, PeriodicBox};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let cell = PeriodicBox::new([10.0, 8.0])?;
    assert_eq!(cell.wrap([-1.0, 9.0])?, [9.0, 1.0]);
    assert_eq!(cell.displacement([0.0; 2], [5.0, 0.0])?,
               [-5.0, 0.0]);
    let lattice = Lattice2D::new(3, 4, Boundary::Periodic)?;
    assert_eq!(lattice.len(), 12);
    assert_eq!(lattice.bonds().count(), 24);
    assert_eq!(lattice.neighbors(0)?.len(), 4);
    Ok(())
}
```

Listing 3.2: Periodic geometry and unique lattice bonds.

3.3 Lattice topology

`Lattice2D` stores a rectangular, row-major topology with open or periodic boundaries. Open dimensions may be one; periodic dimensions must both be at least three. This excludes ambiguous repeated-bond conventions on one- or two-site periodic directions. Dimensions must be nonzero and their checked product must fit the implementation’s indexing range.

`neighbors(site)` returns up to four neighbors and rejects invalid indices. `bonds()` emits each undirected nearest-neighbor pair once. This convention sets the energy normalization used by the built-in two-dimensional models. The topology supports neither arbitrary graphs nor three-dimensional lattices.

Implementation: `csta_core/src/vec2.rs`.

Implementation: `csta_core/src/vec3.rs`.

Implementation: `csta_core/src/vec4.rs`.

Implementation: `csta_core/src/geometry.rs`.

Chapter 4

Random construction

4.1 Trait and iterator

`Randomizable::sample` constructs a value from a caller-provided RNG. `MonteCarlo<T, R>` repeatedly calls it and is an infinite iterator. Use `take`, an explicit loop bound, or another terminating consumer. Collecting it without a bound does not terminate.

Built-in scalar `f32/f64` sampling is uniform on $[0, 1)$. Built-in vector coordinates are independent draws from that interval. This samples a cube, not a sphere or a thermal velocity distribution. Tuple implementations cover arities two through eight when each member implements the trait. General integer, container, and constrained distributions are not supplied by a universal blanket implementation.

```
use csta::{MonteCarlo, gaussian, isotropic_direction};
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let rng = StdRng::seed_from_u64(42);
    let values: Vec<f64> = MonteCarlo::new(rng).take(32).collect();
    assert!(values.iter().all(|x| (0.0..1.0).contains(x)));
    let mut rng = StdRng::seed_from_u64(43);
    let velocity = gaussian(&mut rng, 0.0, 1.0)?;
    let direction = isotropic_direction(&mut rng);
    assert!(velocity.is_finite());
    assert!((direction.len() - 1.0).abs() < 1e-14);
    Ok(())
}
```

Listing 4.1: Bounded independent sampling and explicit physical distributions.

`gaussian(rng, mean, scale)` uses Box–Muller sampling with finite mean and finite nonnegative standard deviation. A zero scale returns the mean without consuming randomness. Positive scale uses two draws; no companion Gaussian is cached. A nonfinite result is an error. `isotropic_direction` uses uniform $z \in [-1, 1)$ and azimuth $\phi \in [0, 2\pi)$ to return $(\sqrt{1 - z^2} \cos \phi, \sqrt{1 - z^2} \sin \phi, z)$ on S^2 . Neither helper performs an ensemble equilibration step.

4.2 Deriving random models

The derive supports named, tuple, and unit structs, plus nonempty enums with those variant shapes. Unconfigured fields use `Randomizable`. Generic bounds are generated for sampled or

defaulted field types as needed. Enum variants are uniform unless every variant declares a weight.

Attribute	Meaning and constraint
<code>range(a..b)</code>	Random range, excluding <i>b</i> ; inclusive ranges use <code>a..=b</code> . Invalid literal endpoints are compile errors. Dynamic invalid ranges can panic at sampling time.
<code>len(n)</code>	Construct a <code>Vec</code> of sampled members. The length must convert to <code>usize</code> ; allocation remains the caller's responsibility.
<code>default</code>	Use the field type's <code>Default</code> .
<code>default = expr</code>	Evaluate an explicit initializer, including dependencies on other named fields.
<code>after(expr)</code>	Sample the field first, then evaluate its transformation.
<code>mul, div, add, sub</code>	Arithmetic transforms in attribute order; do not combine with a conflicting base initializer.
<code>weight = literal</code>	Enum variant weight. All variants must declare finite, nonnegative numeric weights, with at least one positive value.

```
use csta::{Randomizable, csta_derive};
use rand::{SeedableRng, rngs::StdRng};

#[derive(csta_derive::Randomizable)]
struct Proposal {
    #[csta(range(-0.5..0.5))]
    displacement: f64,
    #[csta(default = displacement * displacement)]
    squared: f64,
    #[csta(len(3))]
    auxiliary: Vec<f64>,
}

fn main() {
    let p = Proposal::sample(&mut StdRng::seed_from_u64(42));
    assert!((-0.5..0.5).contains(&p.displacement));
    assert_eq!(p.squared, p.displacement * p.displacement);
    assert_eq!(p.auxiliary.len(), 3);
}
```

Listing 4.2: A bounded proposal type constructed by the `derive` macro.

Named-field dependencies are analyzed syntactically and evaluated in dependency order. Cycles, unknown attributes, conflicting initializers, empty enums, and unions produce compiler diagnostics. Avoid shadowing field names inside initializer expressions: the dependency analysis does not perform full Rust name resolution. Attribute expressions can use `rng`.

Weights are rescaled by their maximum to avoid an overflowing total. Zero-weight variants are never selected. Extreme weight ratios can still lose representable probability. The `derive` is an infallible constructor, not a general distribution language or a proof that sampled states satisfy a Hamiltonian's constraints.

Implementation: `csta_montecarlo/src/lib.rs`.

Implementation: `csta_derive/src/lib.rs`.

Implementation: `csta_derive/README.md`.

Chapter 5

The model contract

5.1 State, parameters, and reversible changes

State separates physical configuration from Hamiltonian parameters and proposal descriptions. Both canonical and Wang–Landau sampling use it. A **Change** must be cloneable; parameters used by checked execution must provide an independent **Clone**. The minimum interface is:

```
trait State {  
    type Params;  
    type Change: Clone;  
    fn energy(&self, params: &mut Self::Params) -> f64;  
    fn propose_change(&self, rng: &mut impl rand::RngExt)  
        -> Self::Change;  
    fn apply_change(&mut self, change: Self::Change);  
    fn revert_change(&mut self, change: Self::Change);  
}
```

Listing 5.1: Required trait methods; interface excerpt.

energy may refresh a parameter cache. Rejected moves restore the parameter snapshot. A clone that shares mutable cache storage through interior mutability cannot provide that guarantee. A cache held inside the state must be restored by **revert_change** itself. Store the previous cached value in **Change** when inverse floating-point arithmetic would not restore it exactly.

Parameters describe a fixed Hamiltonian during a sampling run. Mutating physical parameters through an energy cache changes the target distribution and violates the contract. All model methods must terminate. Cancellation is cooperative and cannot preempt a callback that loops indefinitely.

5.2 Optional hooks

`log_proposal_ratio(&self, &change)` returns

$$c(x, y) = \log \frac{q(x \mid y)}{q(y \mid x)}. \quad (5.1)$$

It is evaluated before applying the move and defaults to zero, which assumes symmetric proposals. Negative infinity means zero reverse probability and rejects the move. NaN and positive infinity are invalid. **Hastings**<S,F> supplies the same correction through a closure without changing the wrapped model type’s implementation.

```

use csta::{Hastings, Metropolis, State};
use rand::{RngExt, SeedableRng, rngs::StdRng};

struct Bit(bool);
impl State for Bit {
    type Params = ();
    type Change = (bool, bool);
    fn energy(&self, _: &mut ()) -> f64 { f64::from(self.0) }
    fn propose_change(&self, rng: &mut impl RngExt) -> Self::Change {
        (self.0, rng.random_bool(0.75))
    }
    fn apply_change(&mut self, (_, next): Self::Change) {
        self.0 = next;
    }
    fn revert_change(&mut self, (old, _): Self::Change) {
        self.0 = old;
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let model = Hastings {
        state: Bit(false),
        log_ratio: |_: &Bit, &(old, next): &(bool, bool)| {
            let q = |b| if b { 0.75_f64 } else { 0.25 };
            (q(old) / q(next)).ln()
        },
    };
    let mut run = Metropolis::with_all(
        model, (), 1.0, 2_000, StdRng::seed_from_u64(42),
    );
    assert_eq!(run.run(None)?.attempted, 2_000);
    Ok(())
}

```

Listing 5.2: Correcting an asymmetric independence proposal.

Here the proposal draws the next bit independently with probability 0.75 for true. Its reverse/forward ratio is $q(\text{old})/q(\text{next})$. Including that factor targets the canonical excited probability $1/(1 + e^\beta)$; using zero correction would retain the proposal's bias. A self-loop has correction zero.

`delta_energy` optionally returns a local $H(y) - H(x)$; `None` requests full energy evaluation. Only use it when accepted moves do not require a skipped parameter-cache refresh. `valid_state` defaults to true and is used in checkpoint validation; custom persisted models should override it to check structural invariants. It is not an automatic per-proposal physical validator.

```

use csta::{Metropolis, State};
use rand::{RngExt, SeedableRng, rngs::StdRng};

#[derive(Clone)]
struct TwoLevel { excited: bool }

impl State for TwoLevel {
    type Params = f64; // excitation gap
    type Change = bool; // previous state
    fn energy(&self, gap: &mut f64) -> f64 {
        if self.excited { *gap } else { 0.0 }
    }
    fn propose_change(&self, _: &mut impl RngExt) -> bool {
        self.excited
    }
    fn apply_change(&mut self, old: bool) {
        self.excited = !old;
    }
    fn revert_change(&mut self, old: bool) {
        self.excited = old;
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut run = Metropolis::with_all(
        TwoLevel { excited: false }, 1.0, 0.7, 2_000,
        StdRng::seed_from_u64(42),
    );
    let report = run.run(None)?;
    assert_eq!(report.attempted, 2_000);
    Ok(())
}

```

Listing 5.3: A complete reversible two-level model.

This deterministic toggle proposal is symmetric. At $\beta = 0$ it alternates between states and is periodic; a self-loop proposal would be needed to remove that periodicity. At the positive β shown, energetic rejections provide self-loops. Generic trait conformance does not establish irreducibility, aperiodicity, or adequate exploration.

Implementation: `csta_metropolis/src/lib.rs`.

Chapter 6

Canonical sampling and observations

6.1 Transition and rollback

A canonical proposal uses the log acceptance ratio

$$r = \beta[H(x) - H(y)] + c(x, y), \quad \alpha(x, y) = \min(1, e^r). \quad (6.1)$$

The implementation compares $\log u$ with r and avoids exponentiating it. Finite energy and finite β are required. Overflow of a valid Boltzmann log ratio has its limiting acceptance meaning; an indeterminate ratio is an error. At $\beta = 0$, the Boltzmann part is zero before any potentially overflowing energy subtraction.

`try_step` validates counters, snapshots parameters, evaluates old energy, constructs a proposal and correction, applies it, evaluates the proposed energy, and accepts or restores the move. On a checked proposal error, the state, parameters, and attempt counts are restored under the model contract. Random draws already consumed are not rewound. Panics in user model methods are not a checked transactional error path.

A completed rejection still increments `attempted_moves`. Accepted self-loops count as accepted moves. Lifetime acceptance is accepted divided by attempted; both accepted and rejected rates are zero before any attempts. Public fields allow explicit control, but callers can also invalidate a model, cache, or counter by mutating them directly.

6.2 Budgets and observation timing

`with_all(state, params, beta, steps, rng)` is the most explicit constructor. Convenience constructors fill defaults or use the thread RNG. Constructor names ending in `_no_beta` use $\beta = 1$. Construction itself is not a checked run; execution performs validation.

`run` executes at most `steps` proposals per call and returns a `RunReport` with this call's attempts, accepts, and `Complete` or `Cancelled`. Lifetime counters accumulate across calls. `try_step` returns one checked acceptance decision. Legacy `step`, `run_empty`, and observer conveniences panic on invalid execution instead of returning errors.

A `Schedule` skips `burn_in` completed proposals and then measures every `stride` proposals. The first measurement is at `burn_in + stride`. Stride must be positive, and schedules restart for each run call. Measurements describe the retained state after the move, including repeated states after rejection. Counting accepted states alone would change the sampled measure.

```

use csta::{Metropolis, Schedule, models::Ising2D};
use csta::statistics::Blocking;
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut run = Metropolis::with_all(
        Ising2D::aligned(4)?, (), 0.4, 12_000,
        StdRng::seed_from_u64(42),
    );
    let mut energy = Blocking::new(128)?;
    let report = run.try_run_observed(
        Schedule { burn_in: 2_000, stride: 1 }, None,
        |s, _| energy.push(s.recompute_energy() / 16.0),
    )?;
    assert_eq!(report.attempted, 12_000);
    assert_eq!(energy.moments().count(), 10_000);
    println!("energy/site estimate: {:?}", energy.estimate());
    Ok(())
}

```

Listing 6.1: Streaming canonical observations with constant-memory blocking.

6.3 Observer interfaces

`run_observed` accepts an infallible streaming closure; `try_run_observed` accepts a closure returning the canonical `Result<()>`. A measurement error occurs after a committed move and leaves the run at that move. External output and callback side effects are not rolled back.

`Observer` provides static measurement, interval, and finalization methods. `DynObserver` provides the corresponding instance-based interface. `run_with` returns one collected series; `run_with_2`, `run_with_3`, and `run_with_4` collect heterogeneous observer outputs. `run_with_n` handles dynamic observers with a common output type. These interfaces retain observations in memory; a streaming closure can maintain only moments, blocks, or a file sink.

Burn-in and thinning are controls, not automatic equilibration detection. Acceptance rate does not measure mixing by itself. Observing every proposal is often useful for estimating temporal dependence; changing stride changes the time unit of the resulting autocorrelation estimate.

Implementation: `csta_metropolis/src/lib.rs`.

Implementation: `csta_metropolis/src/observer.rs`.

Chapter 7

Lattice models and temperature exchange

7.1 Two-dimensional Ising model

`models::Ising2D` uses rectangular `Lattice2D` connectivity and spins $s_i \in \{-1, +1\}$:

$$H = -J \sum_{\langle ij \rangle} s_i s_j - h \sum_i s_i, \quad M = \sum_i s_i. \quad (7.1)$$

Each bond appears once. Construction validates finite J, h , spin values, lattice size, and boundary rules. `aligned(L)` is the periodic $L \times L$ model with all spins positive, $J = 1$, and $h = 0$.

A single-site flip has $\Delta H = 2s_i(J \sum_{j \sim i} s_j + h)$. The model caches total energy and records the previous cached value in each change for exact rejection rollback. `recompute_energy` provides an independent full traversal; magnetization also traverses the lattice. Repeated accepted floating-point updates can drift relative to recomputation, so noninteger couplings warrant periodic comparisons in a validation study.

`wolff_step(beta, rng)` is restricted to $J \geq 0$, $\beta \geq 0$, and $h = 0$. It grows a cluster through aligned neighbors with $p = 1 - \exp(-2\beta J)$ and flips that cluster, following the usual ferromagnetic construction [4]. It returns cluster size. At $\beta J = 0$, the cluster is a single spin. The implementation uses lattice-sized scratch storage and recomputes total energy after the update; total work is therefore $O(N_s)$ even when the cluster is small. It has no general field or frustrated-spin extension.

7.2 Lattice gas

`models::LatticeGas` stores binary occupations $n_i \in \{0, 1\}$ on the same topology. Its physical interaction energy is $H = -J \sum_{\langle ij \rangle} n_i n_j$; `State::energy` returns $H - \mu N_p$ with $N_p = \sum_i n_i$. A uniformly chosen site is toggled, so insertion and removal proposals are symmetric on this configuration space. There are no continuum volume, factorial, or particle-selection corrections to add for this specific move.

`physical_energy` excludes the chemical potential. Use it when reporting H , and use the effective energy for acceptance or reweighting at fixed μ . The model has cached reversible energy updates. It does not implement mobile particles, continuum insertions, or a general grand-canonical acceptance helper.

```

use csta::{Metropolis, State, models::{Ising2D, LatticeGas}};
use csta::geometry::Boundary;
use rand::{SeedableRng, rngs::StdRng};
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut rng = StdRng::seed_from_u64(42);
    let mut spins = Ising2D::aligned(4)?;
    let cluster = spins.wolff_step(0.4, &mut rng)?;
    assert!((1..=16).contains(&cluster));
    let gas = LatticeGas::new(4, 4, Boundary::Periodic,
                             1.0, -2.0, vec![false; 16])?;
    let mut run = Metropolis::with_all(gas, (), 0.5, 1_000, rng);
    run.run(None)?;
    println!("effective energy: {}", run.state.energy(&mut ( )));
    println!("physical energy: {}", run.state.physical_energy());
    Ok(())
}

```

Listing 7.1: Cluster updates and a grand-canonical lattice gas.

7.3 Temperature replica exchange

`replica::ReplicaExchange` accepts one initial state per finite, strictly increasing inverse temperature, a common parameter value, and a seed. Parameters are independently cloned. Local Metropolis chunks run sequentially; adjacent pairs alternate between two disjoint pairing patterns.

For temperatures β_i, β_j and energies E_i, E_j , the swap log ratio is

$$r_{\text{swap}} = (\beta_i - \beta_j)(E_i - E_j). \quad (7.2)$$

Configurations, parameter caches, and identity labels move between slots. Temperatures, local RNGs, and local move counters remain attached to the slots. The common-Hamiltonian assumption is essential; this is not general Hamiltonian replica exchange.

```

use csta::{models::Ising2D, replica::ReplicaExchange};
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let states = vec![Ising2D::aligned(4)?; 3];
    let mut replicas = ReplicaExchange::new(states, (),
                                           vec![0.2, 0.4, 0.6], 42)?;

    replicas.run(20, 100, None)?; // rounds, local proposals
    println!("swaps: {}/{}", replicas.swap_accepts,
            replicas.swap_attempts);
    println!("round trips: {}", replicas.round_trips);
    Ok(())
}

```

Listing 7.2: An explicit inverse-temperature ladder.

Swap attempts, accepts, and cold-hot-cold round trips are separate diagnostics. A single replica is valid and performs no exchange. A saved local cursor allows cancellation in a chunk to resume using the original `local_steps`; the requested round count includes completion of a pending round. A whole round is not an atomic transaction. Direct `exchange(i)` calls do not perform the round-trip bookkeeping that the round runner performs.

The ladder is explicit; no automatic optimization or CPU-parallel execution is provided. Poor energy overlap suppresses swaps. Good swap acceptance alone does not establish barrier crossing.

Implementation: `csta_metropolis/src/models.rs`.

Implementation: `csta_metropolis/src/replica.rs`.

Chapter 8

Statistics and finite-size analysis

8.1 Streaming moments

Moments stores count, mean, and centered second-moment sum. Welford-style updates and Chan-style merges avoid direct subtraction of large raw second moments. **variance** uses $n - 1$ and needs at least two samples; **population_variance** uses n and needs one. Empty means are **None**. Nonfinite observations, numerical overflow, and counter overflow are errors; failed updates leave the previous accumulator intact.

Covariance similarly accumulates paired scalar observations and returns sample covariance for $n > 1$. These accumulators estimate moments, not independent sample counts. Correlated observations require an additional error estimator.

8.2 Blocking and autocorrelation

Blocking uses nonoverlapping fixed-size blocks and constant memory. It requires at least eight complete blocks and positive block-mean variance before returning an estimate. The standard error is

$$\text{SE}(\bar{A}) = \sqrt{\frac{s_{\text{block means}}^2}{B}}, \quad \text{ESS} = \min \left(\frac{s_{\text{completed observations}}^2}{\text{SE}(\bar{A})^2}, Bb \right), \quad (8.1)$$

where b is block length and B is the number of complete blocks. An incomplete trailing block is excluded from the estimate's mean, uncertainty, and sample count. **moments()** still includes every observation. The default block length is 64; no automatic choice or decorrelation guarantee is implied.

correlated_estimate analyzes an explicitly retained scalar series. It uses positive adjacent autocovariance pairs with a nonincreasing constraint, in the initial-sequence family [3]. With lag autocorrelation ρ_k , its convention is $\tau = 1 + 2 \sum_{k>0} \rho_k$, so $\text{ESS} = n/\tau$ and $\text{SE}^2 \approx \gamma_0 \tau / n$. This τ is twice the integrated time under the alternative $\frac{1}{2} + \sum \rho_k$ convention.

The implementation floors τ at one. It requires at least 32 nonconstant observations, a resolved nonpositive pair before the lag cap, and $n \geq 20\tau$. The cap is $\min(n/2, 4096)$; unresolved series return **None**. Work is $O(n \min(n/2, 4096))$. Its assumptions concern stationary reversible chains. It does not diagnose equilibration, and should not be applied uncritically to adaptive WL output or arbitrary deterministic dynamics.

8.3 Thermodynamic summaries and nonlinear errors

`Thermodynamics::push(E,M)` consumes total energy and signed total magnetization. With N_s sites, `summary(beta,k_b,sites)` computes

$$e = \langle E \rangle / N_s, \quad m = \langle M \rangle / N_s, \quad (8.2)$$

$$c = k_B \beta^2 \text{Var}(E) / N_s, \quad \chi = \beta \text{Var}(M) / N_s, \quad (8.3)$$

$$U_4 = 1 - \frac{\langle M^4 \rangle}{3 \langle M^2 \rangle^2}. \quad (8.4)$$

The variances in these fluctuation formulas use population moments. The Binder cumulant is `None` when its denominator vanishes. The API requires finite β , positive finite k_B , nonzero site count, and at least one observation. It does not replace M with $|M|$ or infer a symmetry-breaking convention.

`thermal_errors` applies a delete-one-block jackknife to these nonlinear summaries. It needs at least eight complete blocks, accepts at most 4096 blocks, and discards trailing incomplete samples. Combining leave-one-block summaries costs $O(B^2)$ after reading the observations. Constant complete data can return `None`; blocks must be chosen longer than the relevant correlation scale. The summary alone provides no uncertainty estimate.

```
use csta::{State, models::Ising2D};
use csta::statistics::{Thermodynamics, thermal_errors};
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut model = Ising2D::aligned(4)?;
    let mut rng = StdRng::seed_from_u64(42);
    let mut moments = Thermodynamics::default();
    let mut pairs = Vec::new();
    for i in 0..5_000 {
        model.wolff_step(0.4, &mut rng)?;
        if i >= 1_000 {
            let pair = (model.energy(&mut ()), model.magnetization());
            moments.push(pair.0, pair.1)?;
            pairs.push(pair);
        }
    }
    println!("{}", moments.summary(0.4, 1.0, 16)?);
    println!("{}", thermal_errors(&pairs, 0.4, 1.0, 16, 128)?);
    Ok(())
}
```

Listing 8.1: Thermodynamic summaries with an explicit size and block length.

8.4 Finite-size helpers

`finite_size::crossings` linearly interpolates every crossing of two curves on an exactly shared, finite, strictly increasing coordinate grid. Empty and multiple results remain explicit; a coincident segment is ambiguous and returns an error. `peak` returns a unique observed interior maximum, or `None` for a tie or boundary maximum. Neither routine fits a scaling law or extrapolates.

`block_bootstrap` resamples complete nonoverlapping blocks with replacement and evaluates a user-supplied scalar statistic. It returns the bootstrap standard deviation. It requires a positive block size, at least eight blocks, at least two repetitions, and finite data; incomplete trailing

samples are excluded. Choosing a block length and a physically appropriate statistic remains the caller's task.

```
use csta::finite_size::{crossings, peak};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let a = [(0.0, 0.0), (1.0, 1.0), (2.0, 0.0)];
    let b = [(0.0, 0.5), (1.0, 0.5), (2.0, 0.5)];
    assert_eq!(crossings(&a, &b)?, vec![0.5, 1.5]);
    assert_eq!(peak(&a)?, Some((1.0, 1.0)));
    assert!(peak(&b)?.is_none());
    Ok(())
}
```

Listing 8.2: Explicit crossings and peaks on a finite grid.

For a finite-size study, collect independent or appropriately blocked curves for several sizes, retain all crossings, and vary the temperature grid and block length. A finite-size susceptibility peak or Binder crossing is not itself a thermodynamic-limit critical point. These helpers provide neither critical exponents nor automated model selection, fit covariance, or plotting.

Implementation: `csta_metropolis/src/statistics.rs`.

Implementation: `csta_metropolis/src/finite_size.rs`.

Chapter 9

Deterministic dynamics

9.1 A separate NVE interface

`dynamics::Dynamics<D>` stores vectors of positions, velocities, and positive masses. D must be positive, particle arrays must be nonempty and equally sized, and all coordinates and velocities must be finite. The API uses arrays [f64; D] rather than the small-vector types. It does not require `State`, an RNG, or a Monte Carlo acceptance rule.

A force callback maps positions to a potential energy and one force vector per particle. Forces must be consistent with the potential and be pure functions of the supplied coordinates. Fixed-step velocity-Verlet [5] advances

$$\mathbf{v}_{i,n+1/2} = \mathbf{v}_{i,n} + \frac{\Delta t}{2m_i} \mathbf{F}_i(\mathbf{r}_n), \quad (9.1)$$

$$\mathbf{r}_{i,n+1} = \mathbf{r}_{i,n} + \Delta t \mathbf{v}_{i,n+1/2}, \quad (9.2)$$

$$\mathbf{v}_{i,n+1} = \mathbf{v}_{i,n+1/2} + \frac{\Delta t}{2m_i} \mathbf{F}_i(\mathbf{r}_{n+1}). \quad (9.3)$$

The step size must be finite and positive. Two force evaluations and a cloned candidate particle state are used per step; forces are not cached between calls. A failed step leaves the original particles unchanged, but cannot undo external callback effects. A failed multi-step run retains earlier successful steps.

9.2 Energy monitoring and force helpers

`run` returns initial energy, final energy, and maximum absolute drift from the initial value. Absolute drift avoids division by zero. Zero steps are valid with a valid time step and initial energy. A finite trajectory can still be unstable or inaccurate; users must compare time steps and impose a suitable drift tolerance. An NVE trajectory is not automatically an ergodic sample of an energy surface.


```

use csta::dynamics::{Dynamics, harmonic};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut particles = Dynamics::new(
        vec![[1.0]], vec![[0.0]], vec![[1.0]],
    )?;
    let drift = particles.run(1_000, 0.001, |x| harmonic(x, 1.0))?;
    assert!(drift.maximum_absolute < 1e-6);
    assert!((particles.positions()[0][0] - 1.0_f64.cos()).abs()
        < 1e-6);
    Ok(())
}

```

Listing 9.1: A harmonic trajectory with an explicit drift check.

`harmonic` implements $U = \frac{1}{2}k \sum_i |\mathbf{r}_i|^2$ with finite $k \geq 0$. `lennard_jones` evaluates all nearest-image pairs in a supplied orthorhombic box using

$$U = 4\epsilon \sum_{i < j} \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right]. \quad (9.4)$$

It requires positive finite ϵ, σ and rejects coincident or unrepresentable pair separations. Pair forces are equal and opposite. Positions remain unwrapped in storage; the force helper applies nearest images.

The Lennard–Jones helper is a tiny-system example with $O(N_p^2 D)$ work. It has no cutoff, neighbor list, tail correction, electrostatics, or full periodic-image sum. Minimum-image branch changes also require care for smooth dynamics. No thermostat, barostat, constraints, adaptive time step, particle checkpoint API, or production bulk-fluid accuracy claim is provided.

Implementation: `csta_core/src/dynamics.rs`.

Chapter 10

Energy grids and DOS data

10.1 Explicit support

`EnergyGrid::discrete(levels)` requires a nonempty sequence of finite, strictly increasing energies with representable adjacent differences. Lookup is exact. A single level is valid. Integer-valued energies represented exactly in `f64` are convenient; a numerically rounded Hamiltonian must not rely on a nearby level being accepted.

`EnergyGrid::continuous(min,max,bins)` requires finite $\text{min} < \text{max}$, positive bin count, finite positive width, and distinct representable edges. Intervals are half-open, except that the last includes the upper endpoint. Thermodynamic calculations use stored bin centers. Finite energies outside the interval return `None`; NaN and infinities return an error. The implementation does not clamp energies into endpoint bins.

```
use csta::wl::{EnergyGrid, RawWangLandauData};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let continuous = EnergyGrid::continuous(-1.0, 1.0, 4)?;
    assert_eq!(continuous.bin(-1.0)?, Some(0));
    assert_eq!(continuous.bin(1.0)?, Some(3));
    assert_eq!(continuous.bin(1.01)?, None);
    assert!(continuous.bin(f64::NAN).is_err());
    let grid = EnergyGrid::discrete(vec![-2.0, 0.0, 2.0])?;
    let data = RawWangLandauData::with_support(
        grid, vec![true, false, true],
        vec![0.0, f64::NEG_INFINITY, 0.0],
    );
    assert_eq!(data.energy_to_bin(0.0)?, None);
    assert!(data.is_flat());
    Ok(())
}
```

Listing 10.1: Endpoint behavior and an explicit inaccessible bin.

On an existing grid, `on_grid` initially gives every bin zero log-DOS. The explicit constructor `with_support` takes a grid, mask, and log masses: accessible entries must be finite, inaccessible entries must be negative infinity, and at least one entry must be accessible. All arrays must match the grid. The sampler never infers inaccessible states from missing visits.

A support mask can disconnect proposal paths. Declaring both endpoints of a path accessible does not create a proposal connecting them. Conversely, an omitted physical energy changes the ensemble being estimated. Support design therefore requires model knowledge, not histogram

inspection alone.

10.2 Mass, density, and normalization

DOS arrays contain $\ell_i = \ln g_i$. For a discrete level, g_i is a state count or degeneracy. For a continuous interval I_i , the intended quantity is bin mass, $g_i = \int_{I_i} g(E) dE$. The partition sum uses g_i directly; an additional bin-width factor would count the width twice.

Sampling determines relative log-DOS values, leaving an additive constant undetermined. `normalize_log_count(ln_total)` sets the log of total mass; `anchor(i, ln_degeneracy)` sets one finite bin. Relative probabilities are unchanged. Absolute free energy and entropy require a physically meaningful normalization. A continuous model also needs a measure convention; the library does not supply a phase-space cell, indistinguishability factor, or quantum scale.

Continuous-bin quadrature replaces energy within a bin by its center and assumes constant DOS for acceptance within that bin. Binning error remains even after arbitrarily long sampling. Vary bin width and support bounds separately from the sampling budget. There is no automatic binning, adaptive grid, or interpolation inside a bin.

10.3 Data lifecycle and counters

Raw data holds log-DOS, current-stage histogram, lifetime histogram, support, and their visit totals. Fields are private and constructors validate shape. `clear_hist` clears current counts only. `process_data` consumes raw data into `WLData`, retaining the grid, log-DOS, and current histogram; it does not retain all raw lifetime diagnostics.

A processed `WLData` can also be constructed from exact or externally estimated data. Its histogram is bookkeeping and is not used as an alternative DOS in thermodynamic calculations. Negative-infinite log masses represent absent levels; at least one mass must be finite. Reusing an old DOS with fresh counters starts a new adaptive experiment. It is not exact continuation of a previous sampler.

Implementation: `csta_wl/src/grid.rs`.

Implementation: `csta_wl/src/lib.rs`.

Implementation: `csta_wl/src/wl_data.rs`.

Chapter 11

Serial Wang–Landau and SAMC

11.1 Algorithm composition

The implementation combines Wang–Landau preliminary stages with SAMC production. The production schedule follows the trial-move convention described by Shakirov [2]. For a current bin i and proposed accessible bin j , the log acceptance ratio is

$$r = \ell_i - \ell_j + c(x, y). \quad (11.1)$$

A finite off-support proposal is rejected and the current state is retained. A nonfinite energy is an error with rollback. After every completed proposal, the retained bin b receives

$$\ell_b \leftarrow \ell_b + \gamma, \quad h_b \leftarrow h_b + 1. \quad (11.2)$$

This update also occurs after rejection or a self-loop. The acceptance decision uses the pre-update DOS. A positive update that overflows or rounds back to the old value is rejected as a numerical operation; no visit is then recorded.

11.2 Preliminary refinement

A preliminary stage uses a constant update, initially `initial_ln_f`. With B_a accessible bins, a stage completes only after

$$n_{\text{stage}} \geq \max(\text{min_stage_steps}, B_a \text{ visits_per_bin}) \quad \text{and} \quad h_i \geq p\bar{h} \text{ for every accessible } i. \quad (11.3)$$

The mean $\bar{h}$ excludes masked bins and must be positive. Equality satisfies the flatness threshold. `visits_per_bin` sets a minimum total stage budget; it does not by itself require that number in every individual bin.

After a stage completes, the update is halved and the current histogram is cleared. State, log-DOS, lifetime visits, and RNG continue. After the requested number of stages, production starts with its own trial counter at zero. Zero preliminary stages explicitly skips this phase.

11.3 Production schedule and bounded execution

Production uses

$$\gamma(t) = \frac{t_0}{t_1 + t}, \quad (11.4)$$

where t is the number of completed production trial moves before the current proposal. Rejections count. It is not visits divided by bin count, physical time, or number of accepted moves. Defaults are $t_0 = B_a$ and $t_1 = 10t_0$.

Configuration field	Default	Range and meaning
<code>preliminary_stages</code>	5	0–1024; final preliminary update must remain positive and representable.
<code>min_stage_steps</code>	1000	Positive u64; lower bound per stage.
<code>visits_per_bin</code>	500	Positive u64; product with accessible bins must not overflow.
<code>flatness</code>	0.8	Finite fraction in $(0, 1]$.
<code>initial_ln_f</code>	1.0	Finite positive preliminary log-DOS increment.
<code>sampling_steps</code>	100000	Production trial target; zero performs no moves, including no warm-up.
<code>max_steps</code>	10000000	Positive total preliminary-plus-production bound.
<code>t0, t1</code>	Derived	Optional finite positive scales with representable positive updates over the requested schedule.

`run` validates the grid, raw-data state, configuration, and initial energy. The initial state must already lie inside accessible support. It requires `State` and independent parameter cloning, but not `Randomizable`. `sample_in_support` is an optional bounded initializer for models that also implement `Randomizable`; attempts must be positive. Failure to find a state returns `InitialStateOutside`. It is unsuitable as an unbounded search for a rare window.

```
use csta::wl::{self, Config, RawWangLandauData, models::Ising};
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let result = wl::run(
        Ising::<4>::new([1; 4])?, (),
        StdRng::seed_from_u64(42),
        RawWangLandauData::on_grid(Ising::<4>::grid())?,
        Config {
            preliminary_stages: 2,
            min_stage_steps: 100,
            visits_per_bin: 20,
            sampling_steps: 20_000,
            max_steps: 100_000,
            ..Config::default()
        }, None,
    )?;
    if !result.is_complete() {
        return Err("partial DOS run; inspect diagnostics".into());
    }
    let mut dos = result.data.process_data()?;
    dos.normalize_log_count(16_f64.ln())?;
    println!("mean energy: {}", dos.energy_moments(0.5)?.0);
    Ok(())
}
```

Listing 11.1: A bounded serial DOS run with an explicit completion check.

11.4 Diagnostics and stopping semantics

TargetReached means the production target was met. **BudgetExhausted** means the total proposal budget stopped the run first. **Cancelled** means cooperative cancellation was observed. These reasons return partial or complete **RunResult** values; invalid model/data/numerical operations return **Err**. If a move reaches the production target at the total-budget boundary, completion takes precedence.

Counter	Included events
<code>proposals, total_visits</code>	Completed preliminary and production proposals.
<code>sampling_steps</code>	Completed production proposals only.
<code>bins, visits</code>	Current preliminary stage or production histogram.
<code>lifetime_bins</code>	All preliminary and production visits by bin.
<code>accepted</code>	Accepted preliminary and production proposals, including accepted self-loops.
<code>mixing_steps</code>	Parallel frozen-DOS moves after local completion only.
Exchange counts	Eligible window-swap trials and successes; separate from all histogram clocks.

Table 11.2: Counts are intentionally distinct. Each completed adaptive proposal has exactly one retained-bin visit.

Final diagnostics include phase, completed stages, last update, and flatness. The final flatness field is recomputed on finishing. For live histogram flatness, call `flat_at` on `session.data()`; the diagnostics field is not a live convergence estimator.

`wang_landau` and `wang_landau2` are compatibility-shaped conveniences using continuous grids and a thread RNG. Their `target_time` is converted to $\lceil \text{target_time} \times B \rceil$ production proposals, and initialization is bounded at 1000 draws. The shorter wrapper uses two preliminary stages and 500 visits-per-bin in the stage minimum. Prefer `run` for explicit seeds, support, and total budgets.

11.5 Incremental sessions

`Session::new` creates the same internal walker as a full run. `advance(moves, cancel)` advances by at most that many proposals, retaining RNG, cached energy/bin, DOS, histograms, and adaptation clocks. Zero moves leaves the session unchanged. Cancellation can be resumed; original target and total budget remain fixed. Budget exhaustion cannot be silently extended.

`advance_observed` calls a fallible closure for retained production states only, passing the state and cached energy. Warm-up is excluded; rejections and self-loops are included. A callback error follows a committed move. `finish` turns an unfinished session into an explicit cancelled partial result, rather than presenting it as successful.

Flat preliminary histograms and finite production targets do not certify DOS accuracy. Adaptation changes the transition kernel; ordinary stationary-chain error estimates are not automatically valid for its time series. Examine independent runs, coverage, exact small models, normalization, grid resolution, and budget sensitivity.

Implementation: `csta_wl/src/lib.rs`.

Implementation: `csta_wl/src/session.rs`.

Implementation: `csta_wl/README.md`.

Chapter 12

DOS thermodynamics and reference spectra

12.1 Canonical reconstruction

For representative energies E_i and log masses ℓ_i ,

$$\ln Z(\beta) = \text{LSE}_i(\ell_i - \beta E_i), \quad (12.1)$$

$$p_i(\beta) = \exp(\ell_i - \beta E_i - \ln Z), \quad (12.2)$$

$$\langle E \rangle = \sum_i p_i E_i, \quad C = k_B \beta^2 \sum_i p_i (E_i - \langle E \rangle)^2. \quad (12.3)$$

`WLData` shifts both logarithms and energy references before forming probabilities. Variance uses centered differences, avoiding the cancellation in $\langle E^2 \rangle - \langle E \rangle^2$. The public second moment can still overflow when a centered variance is representable.

`log_sum_exp` returns negative infinity for empty or all-negative-infinite input, positive infinity when a positive-infinite term dominates, and an error for NaN. Valid thermodynamic data requires at least one finite mass. Probability underflow can make negligible weights zero; overflow in required final quantities is an error.

`free_energy2(beta,k_b)` computes $F = -\ln Z/\beta$; `free_energy(log_z,beta,k_b)` accepts a supplied log partition function. Both validate $k_B > 0$, but do not multiply by it again because it is already included in β . Zero β is valid for distributions, moments, and heat capacity, but invalid for free energy. `entropy(E,F,T)` computes $(E - F)/T$ with finite nonzero T ; matching T , β , and normalization is the caller's responsibility.

```

use csta::wl::models::{Ising, Oscillators};
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let ising = Ising::<4>::exact_dos()?;
    let beta = 0.5;
    let probabilities = ising.energy_distribution(beta)?;
    assert!((probabilities.iter().sum::<f64>() - 1.0).abs() < 1e-14);
    let energy = ising.energy_moments(beta)?.0;
    let free = ising.free_energy2(beta, 1.0)?;
    println!("E={energy}, F={free}, S={}",
             ising.entropy(energy, free, 1.0 / beta)?);
    let truncated = Oscillators::<2>::exact_dos(10)?;
    println!("truncated oscillator energy: {}",
             truncated.energy_moments(beta)?.0);
    Ok(())
}

```

Listing 12.1: Exact finite-spectrum thermodynamics without sampling noise.

12.2 Microcanonical quantities

`microcanonical_entropy(k_b)` returns $S_i = k_B \ell_i$. `microcanonical_temperature(k_b)` estimates $T(E) = [k_B d\ell/dE]^{-1}$ with a three-point derivative supporting nonuniform spacing. At least three levels are required. Endpoints and stencils containing absent levels return `None`; a zero slope returns positive infinity and a negative slope gives negative temperature. Derivatives amplify DOS noise, and bin-mass entropy depends on the chosen bin convention. This is a finite-difference estimator, not a smoothed microcanonical equation of state.

12.3 Reference models and exact ranges

`wl::models::Ising<N>` is a periodic one-dimensional chain with $J = 1$, $N \geq 3$, and spins ± 1 . Its allowed levels are $E_k = -N + 4k$, $k = 0, \dots, \lfloor N/2 \rfloor$. It caches integer energy and uses reversible single-spin flips. `exact_dos` independently enumerates all 2^N configurations for $3 \leq N \leq 20$, costing $O(N2^N)$ work. It is a reference method, not a large-system solver. This type is distinct from `models::Ising2D` in the canonical module.

`wl::models::Oscillators<N>` represents $N > 0$ distinguishable quantum oscillators in units $\hbar\omega = 1$:

$$E = Q + N/2, \quad Q = \sum_i n_i, \quad n_i \in \{0, 1, \dots\}. \quad (12.4)$$

Stored occupations are `u32`; total quanta are checked and construction rejects totals above 2^{52} to limit loss of exact energy resolution. `grid(Qmax)` truncates the *total* quanta, not each oscillator independently. The exact mass is $g(Q) = \binom{Q+N-1}{N-1}$, accumulated in log form by `exact_dos(Qmax)`. Both exact and sampled thermodynamics must use the same cutoff.

A proposed downward move at occupation zero is a self-loop, preserving symmetry of nontrivial transitions. The random-construction implementation initializes all occupations to zero; it does not draw a thermal or uniform truncated state. Consequently, `sample_in_support` cannot find an oscillator window excluding the ground state with this implementation; supply an appropriate state directly.

Implementation: `csta_wl/src/wl_data.rs`.

Implementation: `csta_wl/src/models.rs`.

Chapter 13

Parallel energy-window sampling

13.1 Window and replica layout

`run_parallel`, also exported as `par_wl`, partitions a shared global energy grid into explicit overlapping windows. Each range is half-open in *bin indices*, not energy values. The first starts at zero and the last ends at the global bin count. Consecutive windows must increase both endpoints, extend coverage, and overlap in at least two bins. Empty windows, gaps, nested ranges, and insufficient overlap are invalid.

`walkers_per_window` must be positive. Initial states are ordered by window, then by replica index; each must lie in its own window. All walkers use the same Hamiltonian parameters with independent mutable clones. Every walker owns its state, cached energy/bin, RNG, DOS, histogram, and schedule.

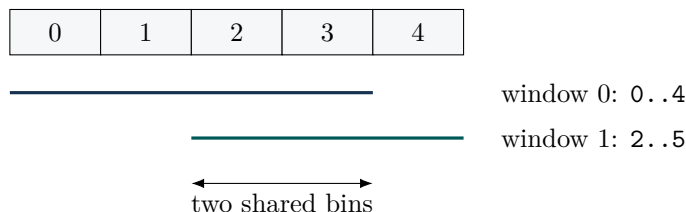


Figure 13.1: An admissible two-window layout for the five levels of an eight-spin periodic Ising chain.

Arrays retain the full global grid in each walker and mask bins outside its window. For R total walkers and B global bins, histogram/DOS storage is $O(RB)$, not the sum of local window lengths. The API has no automatic window placement or memory-budget optimizer.

13.2 Chunk execution and exchanges

Rayon executes indexed walkers in bounded chunks. `exchange_every` is a positive proposal interval. Seeds are assigned before parallel execution, so successful seeded runs retain the documented reproducibility across Rayon thread counts for the same implementation/platform and independent model behavior. Worker panics become `WorkerFailed`; peer cancellation is checked between proposals. A model callback still cannot be forcibly interrupted.

Alternating disjoint adjacent-window pairs attempt swaps between equal replica indices. A trial is eligible only if both configurations fit the destination supports. Following the window-exchange

construction [1], its log ratio is

$$r_{ij} = \ell_i(E_i) - \ell_i(E_j) + \ell_j(E_j) - \ell_j(E_i). \quad (13.1)$$

State, parameter cache, energy, and current bin travel together. DOS, local RNG, histograms, and adaptation schedules remain in their windows. An exchange does not add a histogram visit or advance the SAMC clock.

A walker stops its adaptive chunk immediately when locally complete. If other walkers remain active in later chunks, it makes frozen-DOS moves that increment only `mixing_steps`. Once all walkers finish, no extra exchange is needed. These counting rules are specific to this implementation and should be retained when comparing with other replica-exchange WL codes.

```
use csta::wl::{run_parallel, Config, ParallelConfig, models::Ising};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let states = vec![
        Ising::<8>::new([1; 8])?,
        Ising::<8>::new([1, -1, 1, -1, 1, -1, 1, -1])?,
    ];
    let result = run_parallel(
        Ising::<8>::grid()?, states, (),
        ParallelConfig {
            windows: vec![0..4, 2..5],
            walkers_per_window: 1,
            exchange_every: 100,
            seed: 42,
            run: Config {
                preliminary_stages: 0,
                sampling_steps: 30_000,
                max_steps: 30_000,
                ..Config::default()
            },
        }, None,
    )?;
    let mut dos = result.merged.ok_or("no complete merged DOS")?;
    dos.normalize_log_count(256_f64.ln())?;
    println!("mean energy: {}", dos.energy_moments(0.5)?.0);
    Ok(())
}
```

Listing 13.1: Two overlapping windows with explicitly valid initial states.

13.3 Averaging and joining DOS fragments

After completion, replicas within a window are anchored to a common bin and their log-DOS values are averaged. Each accessible level must have a lifetime visit in every contributing replica. Adjacent window estimates are joined at the overlap point minimizing their entropy-slope mismatch, aligned there, and pasted. Endpoint slopes use secants; interior slopes use nonuniform three-point derivatives. The construction follows the documented joining approach of Moreno, Peralta, and Davis [1].

Merged histograms sum actual current-stage counts from all contributing windows and replicas, independent of the paste point. They do not count only the selected DOS fragment. Missing visits, incompatible grids, or insufficient slope support produce an error, rather than a fabricated merged spectrum.

`ParallelResult::merged` is present only if every walker reaches a nonzero production target

and joining succeeds. Cancellation and budget exhaustion retain explicit partial walkers with no merged DOS; a worker or joining failure returns an error. A successful merge establishes structural compatibility, not small statistical error or absence of a join artifact.

13.4 Parallel sessions and practical limits

`ParallelSession<S,R>` exposes incremental `advance(chunks, cancel)` and `finish`. Its unit is an exchange chunk, unlike serial session proposals. Successful calls end at deterministic exchange barriers, suitable for exact checkpoint continuation. A failed session is not reusable as a valid checkpoint. Asynchronous cancellation can stop different walkers at different proposals; resuming such a partial chunk need not reproduce an uninterrupted exchange schedule.

This is shared-memory parallelism through Rayon. There is no MPI distribution, GPU kernel, distributed checkpoint coordinator, or automatic load balancing of energy-window difficulty. Replicas coupled by exchanges are not independent DOS experiments for uncertainty estimation. Inspect coverage, exchange counts, window disagreement, and independent complete runs.

Implementation: `csta_wl/src/par_wl.rs`.

Implementation: `csta/examples/parallel_ising.rs`.

Chapter 14

Observables, reweighting, and DOS uncertainty

14.1 Conditional observable moments

An energy-only DOS does not determine an arbitrary observable. For $A(x)$,

$$\langle A \rangle_\beta = \sum_i p_i(\beta) \langle A \mid E \in I_i \rangle. \quad (14.1)$$

`analysis::ConditionalMoments` stores per-bin means of A and A^2 . `record(energy,value)` validates the energy, value, and squared value and updates atomically. `evaluate(dos,beta)` requires exactly compatible grids and conditional observations on every finite-DOS bin, even if that bin's canonical probability underflows at the requested temperature.

`Session::advance_observed` connects production states to this accumulator without a new observer trait. It includes repeated retained states. Within-bin samples must follow the intended conditional measure; adaptive transients and poor mixing can bias them. Continuous-bin conditional reweighting additionally approximates variation inside each interval.

```

use csta::wl::{Config, RawWangLandauData, Session, models::Ising};
use csta::wl::analysis::ConditionalMoments;
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let grid = Ising::<4>::grid()?;
    let mut moments = ConditionalMoments::new(grid.clone());
    let mut session = Session::new(
        Ising::<4>::new([1; 4])?, (), StdRng::seed_from_u64(42),
        RawWangLandauData::on_grid(grid)?,
        Config { preliminary_stages: 0, sampling_steps: 20_000,
            ..Config::default() },
    )?;
    session.advance_observed(20_000, None, |state, energy| {
        let m: f64 = state.spins().iter().map(|s| f64::from(*s)).sum();
        moments.record(energy, m)
    })?;
    let result = session.finish();
    if !result.is_complete() { return Err("partial run".into()); }
    let dos = result.data.process_data()?;
    println!("magnetization moments: {:?}", moments.evaluate(&dos, 0.5)?);
    Ok(())
}

```

Listing 14.1: Recording an observable during a serial WL session.

14.2 Single-histogram canonical reweighting

`reweight(samples, source_beta, target_beta)` accepts pairs (E_k, A_k) from a stationary source canonical ensemble. It forms normalized weights

$$w_k \propto e^{-(\beta_t - \beta_s)E_k}, \quad \widehat{\langle A \rangle}_t = \sum_k \tilde{w}_k A_k, \quad \text{ESS}_w = \frac{1}{\sum_k \tilde{w}_k^2}. \quad (14.2)$$

An energy reference is subtracted and weights are normalized logarithmically. Input must be nonempty and finite; unrepresentable log weights are errors. At equal temperatures all weights are equal and the weighted mean is the ordinary sample mean.

The result marks overlap as **Low** if $\text{ESS}_w < 20$ or $\text{ESS}_w < 0.1n$. **Adequate** only means these thresholds passed. Weight ESS detects weight concentration; it does not detect temporal correlation or states absent from the source sample. It is not interchangeable with the autocorrelation ESS from Chapter 8.

```

use csta::{Metropolis, Schedule, models::Ising2D};
use csta::wl::analysis::reweight;
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut run = Metropolis::with_all(
        Ising2D::aligned(4)?, (), 0.4, 12_000,
        StdRng::seed_from_u64(42),
    );
    let mut samples = Vec::new();
    run.run_observed(Schedule { burn_in: 2_000, stride: 1 }, None,
        |s, _| samples.push((s.recompute_energy(), s.magnetization()))?);
    let estimate = reweight(&samples, 0.4, 0.41)?;
    println!("{}", estimate);
    Ok(())
}

```

Listing 14.2: Reweighting a nearby temperature from retained canonical samples.

The helper changes temperature for a fixed effective Hamiltonian. It does not implement WHAM, MBAR, automatic temperature-range selection, or general parameter reweighting with missing sufficient observables. Joint DOS provides an explicit field-dependent alternative in Chapter 15.

14.3 Independent DOS experiments

`IndependentDos::from_run(id,result)` requires a complete run with nonzero production. `from_data` allows external or exact data and assigns responsibility for completion and independence to the caller. Unique IDs prevent including the same labeled run twice; they cannot establish statistical independence or detect duplicated data under different IDs.

`DosEnsemble::new(runs,log_count)` requires matching grids and finite-support patterns and normalizes each run to the same total count. It evaluates an observable on each normalized DOS, then returns its mean and $SE = \sqrt{s^2/R}$ across R runs. One run has no standard error. `log_dos_uncertainty` reports per-bin disagreement, with absent bins left undefined. This preserves nonlinear observable evaluation per experiment rather than treating all walkers as independent observations of one averaged DOS.

```

use csta::wl::{self, Config, RawWangLandauData, models::Ising};
use csta::wl::analysis::{DosEnsemble, IndependentDos};
use rand::{SeedableRng, rngs::StdRng};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut runs = Vec::new();
    for id in 0..3 {
        let result = wl::run(
            Ising::<4>::new([1; 4])?, (),
            StdRng::seed_from_u64(42 + id),
            RawWangLandauData::on_grid(Ising::<4>::grid())?,
            Config { preliminary_stages: 0, sampling_steps: 10_000,
                ..Config::default() }, None,
        )?;
        runs.push(IndependentDos::from_run(id, result)?);
    }
    let ensemble = DosEnsemble::new(runs, 16_f64.ln())?;
    println!("{}", ensemble.energy(0.5)?);
    Ok(())
}

```

Listing 14.3: Uncertainty across separately seeded DOS experiments.

Between-run scatter does not include a bias shared by every run, an incorrect support declaration, or finite-bin quadrature error. There is no automatic independence test, confidence-interval calibration, or convergence certificate.

Implementation: `csta_wl/src/analysis.rs`.

Chapter 15

Sparse joint density of states

15.1 Coordinates and field reweighting

`joint::JointGrid` declares a sparse list of accessible (E_0, M) cells, where E_0 excludes an external magnetic field. Cells must be finite, unique, lexicographically sorted, nonempty, and no more numerous than the supplied `max_cells`. This checks the declared cell count; a caller can already have allocated an oversized vector before invoking the constructor.

`JointDos` requires one finite log mass on each declared cell. Missing cells are outside support, not entries inferred from a zero histogram. For finite β, h , evaluation uses

$$Z(\beta, h) = \sum_{E_0, M} g(E_0, M) e^{-\beta(E_0 - hM)}. \quad (15.1)$$

It returns log partition function, mean physical energy $E_0 - hM$, total magnetization, and cell probabilities. Normalization fixes the total state count. `energy_marginal` sums over M with log-sum-exp and produces an energy DOS in E_0 , appropriate to the zero-field projection.

```
use csta::wl::joint::{JointDos, JointGrid};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let grid = JointGrid::new(vec![(0.0, -1.0), (0.0, 1.0)], 2)?;
    let dos = JointDos::new(grid, vec![0.0, 0.0])?;
    let beta = 0.5;
    let field = 0.3;
    let thermal = dos.evaluate(beta, field)?;
    assert!((thermal.magnetization - (beta * field).tanh()).abs()
        < 1e-14);
    assert!((thermal.energy + field * thermal.magnetization).abs()
        < 1e-14);
    assert_eq!(dos.energy_marginal()?.grid().len(), 1);
    Ok(())
}
```

Listing 15.1: An exact one-spin joint spectrum and field response.

15.2 Joint sampling through the existing engine

`run_joint` takes a `State`, parameters, RNG, declared cells, a read-only magnetization closure, ordinary WL configuration, and optional cancellation. The state's energy must be E_0 . An adapter maps each exact (E_0, M) cell to an index represented as a discrete pseudo-energy; the

existing WL engine then flattens those cells. Finite undeclared cells are rejected; nonfinite coordinates are errors. The proposal-ratio hook is forwarded.

The adapter evaluates the underlying energy instead of forwarding its optional local-energy shortcut. Its cell search costs $O(\log K)$ for K declared cells, and DOS storage is $O(K)$. A large state space can remain impractical even when sparse. No automatic reachability discovery, adaptive cell insertion, continuous multidimensional interpolation, or generalized collective-variable framework is implemented.

The returned `JointRun` contains original state, parameters, joint DOS, and WL diagnostics, with the same explicit completion rule. It does not expose a full per-cell histogram API or a dedicated serializable joint session. Field responses beyond the returned means can be computed from cell probabilities, but joint heat-capacity and susceptibility convenience methods are not supplied.

```
use csta::State;
use csta::wl::{Config, models::Ising};
use csta::wl::joint::{JointGrid, run_joint};
use rand::{SeedableRng, rngs::StdRng};

fn magnetization(s: &Ising<3>) -> f64 {
    s.spins().iter().map(|x| f64::from(*x)).sum()
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut cells = Vec::new();
    for bits in 0..8 {
        let spins = std::array::from_fn(|i| {
            if bits & (1 << i) == 0 { -1 } else { 1 }
        });
        let state = Ising::<3>::new(spins)?;
        cells.push((state.energy(&mut ()), magnetization(&state)));
    }
    cells.sort_by(|a, b| a.partial_cmp(b).unwrap());
    cells.dedup();
    let mut result = run_joint(
        Ising::<3>::new([1; 3])?, ((), StdRng::seed_from_u64(42)),
        JointGrid::new(cells, 8)?, magnetization,
        Config { preliminary_stages: 0, sampling_steps: 20_000,
            ..Config::default() }, None,
    )?;
    if !result.is_complete() { return Err("partial joint run".into()); }
    result.data.normalize_log_count(8_f64.ln())?;
    println!("{:?}", result.data.evaluate(0.5, 0.2)?);
    Ok(())
}
```

Listing 15.2: Sampling explicitly enumerated joint support for a three-spin chain.

Enumeration here establishes support independently of sampling; it is practical only for this tiny system. For a larger model, derive accessible cells from its constraints or another validated support construction.

Implementation: `csta_wl/src/joint.rs`.

Chapter 16

Persistence and reproducible continuation

16.1 Opt-in serialization

Enable `checkpoint` for sampler save/load methods. Supported types are `Metropolis`, serial and parallel WL sessions, and temperature replica exchange. State, parameters, and RNG must implement the required `Serde` traits; parameters must still clone independently. Built-in lattice and WL reference models provide feature-gated support, subject to `Serde` support for their concrete array sizes.

`CheckpointRng` is PCG64 from pinned `rand_pcg` 0.10.2. Use it when a serializable random stream is required. The default `StdRng` is not supported by these serialization bounds. For temperature exchange, use `ReplicaExchange::<_, CheckpointRng>::seeded`; for parallel WL, choose the RNG type on `ParallelSession`. Merely enabling the feature does not make every RNG serializable.

```
use csta::{CheckpointRng, Metropolis, models::Ising2D};
use rand::SeedableRng;

fn main() -> Result<(), Box<dyn std::error::Error + Send + Sync>> {
    let path = std::env::temp_dir().join(format!(
        "csta-design-{}.json", std::process::id()));
    let mut run = Metropolis::with_all(
        Ising2D::aligned(4)?, (), 0.4, 100,
        CheckpointRng::seed_from_u64(42),
    );
    run.run(None)?;
    run.save(&path, "ising-j1-h0-v1")?;
    let mut resumed = Metropolis::<Ising2D, CheckpointRng>::load(
        &path, "ising-j1-h0-v1",
    )?;
    resumed.run(None)?;
    assert_eq!(resumed.attempted_moves, 200);
    std::fs::remove_file(path)?;
    Ok(())
}
```

Listing 16.1: Saving and continuing a canonical run; requires the checkpoint feature.

16.2 File format and validation

The generic core envelope stores format version 1, an implementation tag, caller model tag, Rust type name, and serialized value. The implementation tag is `csta-3/checkpoint-1/rand-0.10.2`. Model tags must be nonempty on save and should change with a Hamiltonian or model schema change. Type names and tags detect mismatches; they do not independently prove physical compatibility.

Saving serializes the value to JSON in memory, creates a sibling temporary file, writes and synchronizes it, renames it to the destination, and synchronizes the parent directory. Temporary-name collisions are retried with a bounded count. Pre-save validation and serialization failures leave an existing destination untouched. A filesystem error after rename can be reported even though the replacement already occurred; the operation is not a multi-file transaction.

Loading reads at most 256 MiB plus one byte to detect an oversized file. It checks envelope compatibility and then sampler-specific invariants: grid shape, support, counters, phase/schedule consistency, and cached state energy. `State::valid_state` supplements energy checks for structural constraints. The low-level generic core loader does not itself know a model's physical invariants. DOS values use floating bit patterns to preserve negative infinity and finite values exactly; JSON float round-trip support preserves other stored floating values.

Files are intended as trusted local snapshots. There is no cryptographic integrity check, signature, compression, cross-version migration, or arbitrary schema conversion. The input-byte limit is not a general cap on all allocations made during deserialization. Large simulations may exceed the current persistence format's practical range.

16.3 What continuation preserves

Serial WL snapshots retain state, parameter cache, RNG, grid, DOS, both histograms, bin/energy cache, preliminary stage, SAMC clock, target, and total budget. Parallel snapshots also retain exchange RNG, parity, and window state at successful barriers. Temperature replicas retain labels and pending local-chunk position. Canonical snapshots retain configured per-call steps and lifetime counters; the next `run` starts another configured budget, not an inferred remainder of a previous call.

External observation vectors, file cursors, closures, and application-level analysis are not automatically saved. Coordinating their state with the sampler is required to avoid duplicated or missing measurements after a restart. Measurement schedules that restart per call also need deliberate handling by the application.

Exact continuation is scoped to the same implementation, dependencies, platform, and deterministic model/callback behavior. Restoring a saved RNG cannot undo nondeterministic external input or shared mutable aliases. Successful parallel barriers provide a stronger reproducibility boundary than asynchronous mid-chunk cancellation. A previous DOS plus fresh counters is a new run, regardless of whether its initial numerical values match a snapshot.

Implementation: `csta_core/src/checkpoint.rs`.

Implementation: `csta_montecarlo/src/lib.rs`.

Implementation: `csta_wl/src/session.rs`.

Implementation: `csta_wl/src/par_wl.rs`.

Implementation: `csta_metropolis/src/replica.rs`.

Chapter 17

Validation, cost, and design boundaries

17.1 Error and partial-result policy

Interface	Failure meaning and boundary
Vector arithmetic	IEEE results; checked direction is <code>Option</code> , not a universal checked-math layer.
Geometry/dynamics	Static-string errors for invalid dimensions, nonfinite coordinates, invalid forces, or numerical failures. One dynamics step commits only after validation.
Random construction	<code>Randomizable</code> is infallible. Dynamic invalid ranges, invalid const-generic reference-model sizes, and allocation failures can panic.
Canonical checked API	<code>Error(&'static str)</code> ; proposal errors restore state, parameters, and counters under the model contract. RNG is not rewound.
Canonical legacy API	Convenience execution can panic. Use <code>try_step</code> or checked run methods when errors must be handled.
Streaming callback	Failure occurs after the observed move commits; external side effects are not transactional.
Statistics	Invalid/numerically unrepresentable observations give errors; short, constant, or unresolved uncertainty data can give <code>None</code> .
Serial WL	Typed invalid-input, energy, support, counter, and numerical errors; cancellation/budget stops return explicit partial results.
Parallel WL	Worker panic or joining failure is an error. Partial sampling has no apparently successful merged DOS. Model hangs cannot be preempted.
Checkpoint	Boxed I/O, format, or validation errors; model tag and structural validation remain necessary.

17.2 Operation and memory costs

Let N_s be lattice sites, N_p particles, D dimension, B energy bins, R total WL walkers, n observations, and C_P the cost of cloning parameters. The table describes the implementation,

excluding user callback cost unless stated. These are operation counts, not measured throughput claims.

Operation	Work	Additional storage
Small vectors	Constant dimension	Constant
Periodic wrap/displacement	$O(D)$	$O(D)$ output
Ising local proposal	$O(1) + C_P$ with cached energy	Change plus parameter snapshot
Wolff update	$O(N_s)$ including energy recomputation	$O(N_s)$ scratch
Moment/block update	$O(1)$	$O(1)$
Correlation estimate	$O(n \min(n/2, 4096))$	Retained $O(n)$ input
Thermal jackknife	$O(n + B_k^2)$, B_k blocks	$O(B_k)$ summaries plus input
Block bootstrap	$O(R_b n)$ plus statistic cost	$O(n)$ resampled series
Harmonic dynamics step	$O(N_p D)$	$O(N_p D)$ candidate/forces
Lennard–Jones step	$O(N_p^2 D)$	$O(N_p D)$ candidate/forces
Serial WL proposal	Model work, C_P , lookup, update	$O(B)$ retained arrays
Preliminary flatness test	$O(B)$ when minimum visits reached	Constant beyond arrays
Parallel WL storage	Per-walker computation and barriers	$O(RB)$ plus states/caches
DOS thermal evaluation	$O(B)$	$O(B)$ weights/output
Sparse joint lookup	$O(\log K)$ for K cells	$O(K)$ DOS/support
Checkpoint save/load	Linear in serialized size	Full JSON buffer and decoded state

Energy-grid lookup uses ordered searches; continuous grids retain edges rather than silently relying on a rounding-sensitive arithmetic index. Preliminary flatness scans can dominate cheap models after the visit minimum. Parameter cloning is part of every checked proposal; a large cache can erase the benefit of a local energy difference. Profile the actual model before adding complexity. Neither checked sizes nor finite budgets guarantee that arbitrary allocations will fit available memory.

17.3 Existing verification strategy

TEST_PLAN.md maps the workspace’s test groups, including ordinary and edge cases. The current suite combines deterministic unit tests, facade integration, compile-fail macro examples, seeded bounded simulations, exact small-spectrum checks, and an explicitly ignored longer release test. This document does not replace that coverage map or claim every possible physical regime is tested.

Area	Important average and boundary checks
Vectors/geometry	Coordinate-preserving casts, arithmetic agreement, tiny/huge normalization, zero/nonfinite directions, periodic endpoints and half-box ties, unique bonds and invalid dimensions.
Random/derive	Seed repeatability, range bounds, vector/enum construction, zero weights, invalid syntax, dependency cycles, distribution moments and sphere isotropy with explicit tolerances.

Area	Important average and boundary checks
Canonical/models	Exact apply/revert including caches, retained-state counting, observer timing, cancellation, asymmetric proposals, nonfinite energies, zero attempts, Ising local differences, gas energy conventions, cluster limits.
Statistics	Known moments, merge agreement, missing/constant data, complete block handling, correlated synthetic series, nonlinear errors, ambiguous curve crossings and boundary peaks.
Dynamics	Harmonic trajectory and step-size scaling, force finite differences, momentum conservation, invalid masses/time steps, coincident pairs, failed-step rollback.
Serial WL	Exact energy mapping, masks, one visit per completed trial, preliminary/production clocks, finite off-support rejection, numerical failure, zero target, cancellation, and total-budget exhaustion.
DOS analysis	Exact finite spectra, stable shifted weights, normalization, absent levels, negative/zero beta, conditional support, weight overlap, duplicate independent-run IDs, and joint-field response.
Parallel/checkpoint	Window validation, exchange counts, frozen mixing, thread-count reproducibility, incomplete merge rejection, worker errors, continuous versus resumed trajectories, invalid formats/tags/caches, and file replacement behavior.

Run repository checks with the required toolchain:

```
cargo fmt --all -- --check
cargo check --workspace --all-targets --all-features
cargo test --workspace
cargo test --workspace --all-features
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test -p csta_wl --release -- --ignored
```

Listing 17.1: Workspace validation from the repository root.

The final command is a deliberate longer stochastic check. Use fixed seeds, finite budgets, exact references where practical, and explicit tolerances. Repeated success of a finite simulation is not proof of universal convergence. Documentation examples are checked separately against the local facade so that API drift is caught without treating demonstration output as a scientific result.

17.4 Scientific validation beyond API correctness

For canonical simulations, compare independent initial states and seeds, inspect observables with different relaxation scales, vary burn-in and block lengths, and verify exact small-system averages. Check signed and absolute order parameters deliberately. A low error estimate on a trapped trajectory is not a reliable physical uncertainty.

For DOS simulations, validate accessible support and proposal connectivity, compare independently normalized DOS estimates, inspect lifetime coverage and window overlaps, vary preliminary/production budgets, and repeat with narrower continuous bins or wider cutoffs. Compare thermodynamics over the intended β range, where rare levels can receive large canonical weight.

For dynamics, compare numerical forces with potential derivatives, test the conserved quantities appropriate to the force law, halve the time step, and check trajectory and drift sensitivity. Potential correctness and integrator correctness are separate. A stable finite time series does not establish bulk transport properties or a thermodynamic limit.

17.5 Deliberate boundaries and extension rules

The implemented range remains explicit. There is no general tensor algebra, arbitrary graph lattice, triclinic geometry, automatic equilibration detector, universal proposal distribution, WHAM/MBAR solver, critical-exponent fitter, production molecular-dynamics force engine, MPI/GPU backend, or cross-version checkpoint migration. Higher-dimensional joint support and rare-event methods require model-specific choices that the current API does not guess.

An extension should keep the common workflow small. Add it to the owning crate, retain the facade dependency direction, give new hooks defaults where possible, and prefer an explicit module or wrapper to new mandatory model methods. Preserve independent parameter rollback and retained-proposal counting. New samplers need finite execution controls and explicit partial results. New numerics need units, valid input ranges, undefined-output behavior, and reference cases before performance claims.

A proposed optimization must preserve seeded behavior where that behavior is part of the documented contract, or document the change. A new physical model must state its ensemble, boundary convention, proposal symmetry, cache invariant, and energy normalization. Optional capability should not impose additional configuration on existing basic runs.

Implementation: `AGENTS.md`.

Implementation: `TEST_PLAN.md`.

Implementation: `TODO.md`.

Appendix A

Source and example map

A.1 Owning implementation files

Path	Role
csta/src/lib.rs, csta/src/prelude.rs	Facade exports and observer prelude.
csta_core/src/lib.rs	Core module declarations.
csta_core/src/vec2.rs, csta_core/src/vec3.rs, csta_core/src/vec4.rs	Vector types and operators.
csta_core/src/geometry.rs	Periodic boxes and lattice bonds.
csta_core/src/dynamics.rs	NVE state, Verlet, force helpers.
csta_core/src/checkpoint.rs	Feature-gated JSON envelope and atomic replacement.
csta_montecarlo/src/lib.rs	Random trait, iterator, distributions, PCG alias.
csta_derive/src/lib.rs	Macro parsing, dependencies, code generation.
csta_metropolis/src/lib.rs	State and canonical runner.
csta_metropolis/src/observer.rs	Static and dynamic observer contracts.
csta_metropolis/src/models.rs	Ising2D, lattice gas, Wolff.
csta_metropolis/src/statistics.rs	Streaming and correlated statistics.
csta_metropolis/src/finite_size.rs	Curves and block bootstrap.
csta_metropolis/src/replica.rs	Temperature exchange and persistence.
csta_wl/src/grid.rs	Energy support and lookup.
csta_wl/src/lib.rs	Raw data, configuration, walker, run APIs.
csta_wl/src/wl_data.rs	DOS normalization and thermodynamics.
csta_wl/src/models.rs	1D Ising and quantum oscillators.
csta_wl/src/par_wl.rs	Rayon chunks, exchange, joining, parallel session.
csta_wl/src/session.rs	Serial continuation and walker snapshot checks.
csta_wl/src/analysis.rs	Conditional moments, reweighting, independent DOS.

Path	Role
<code>csta_wl/src/joint.rs</code>	Sparse joint support and field reweighting.
<code>csta_wl/src/main.rs</code>	Ising/oscillator DOS demonstration binary.

A.2 Runnable repository examples

Run these from the workspace root. Simulation durations and output volumes vary. The document's shorter embedded examples are independent of these programs.

```
cargo run -p csta --release --example state
cargo run -p csta --release --example rand_derive
cargo run -p csta --release --example ising
cargo run -p csta --release --example physics
cargo run -p csta --release --example hastings
cargo run -p csta --release --example reweight
cargo run -p csta --release --example finite_size
cargo run -p csta --features checkpoint --example checkpoint
cargo run -p csta --release --example parallel_ising
cargo run -p csta --release --example profile_energy
cargo run -p csta_wl --release
cargo run -p csta_wl --release -- qho
```

Listing A.1: Repository examples and demonstration binaries.

`state.rs` demonstrates a custom model; `rand_derive.rs` demonstrates random construction. `physics.rs` combines selected physics tools. `finite_size.rs` emits size/temperature observables and uncertainties. `profile_energy.rs` compares energy-evaluation paths; its timings depend on build profile and hardware. The default WL binary uses an Ising chain; the `qho` argument chooses the truncated oscillator example.

Appendix B

Example and document reproducibility

The standalone programs in this document are marked by **example:** comments in the TeX source. `docs/check_examples.py` extracts only those listings, creates a temporary Cargo package depending on the local facade, and checks or runs them. The checkpoint program is validated with its feature enabled. Trait, manifest, and command excerpts are intentionally excluded from extraction.

```
latexmk -pdf -interaction=nonstopmode -halt-on-error \  
  -outdir=docs/build docs/design.tex  
python3 docs/check_examples.py --offline  
python3 docs/check_examples.py --offline --run
```

Listing B.1: Build and validate the document from the workspace root.

`-offline` is appropriate only when dependencies are already cached. The checker uses temporary sources and a build cache under `target/`; it does not add a workspace member or change the library API. The PDF needs standard LaTeX packages listed in the source preamble. Build byproducts are kept in the ignored `docs/build/` directory. The distributed PDF is `docs/design.pdf`; copy the rebuilt PDF there when updating it.

Embedded assertions check exact API invariants or small analytic values. Simulation examples check finite execution and explicit result handling, not a general convergence tolerance. Their printed values are intentionally not included as published physical measurements.

Bibliography

- [1] F. Moreno, J. Peralta, and S. Davis. *A portable and flexible implementation of the Wang–Landau algorithm in order to determine the Density of States*. arXiv:2103.15028v2 (2021). <https://arxiv.org/abs/2103.15028v2>. Referenced for window exchange and slope-based joining; CSTA’s counters and SAMC composition are specified by its own implementation.
- [2] T. Shakirov. *Flat-histogram algorithms: optimal parameters and extended application*. arXiv:2402.05653v2 (2024). <https://arxiv.org/abs/2402.05653v2>. Referenced for the trial-move SAMC schedule. CSTA does not implement every extension described in the paper.
- [3] C. J. Geyer. *Initial Sequence Estimators*, documentation for the mcmc package, referring to *Practical Markov Chain Monte Carlo*, Statistical Science 7, 473–483 (1992). <https://www.stat.umn.edu/geyer/mcmc/library/mcmc/html/initseq.html>. CSTA uses monotone positive pairs, a finite lag cap, and a conservative time floor.
- [4] W. Janke. *Monte Carlo Simulations in Statistical Physics—From Basic Principles to Advanced Applications*. Lecture chapter (2012). https://www.physik.uni-leipzig.de/~janke/Paper/lviv-ising-lecture-janke_corrected.pdf. Referenced for the ferromagnetic cluster construction.
- [5] LAMMPS developers. *fix nve command*, LAMMPS documentation. https://docs.lammps.org/fix_nve.html. Referenced for the velocity-Verlet integration method. CSTA does not wrap LAMMPS or provide its force-field infrastructure.